

ĐỀ CƯƠNG CHI TIẾT KHOÁ LUẬN TỐT NGHIỆP

HỌC KỲ 1 – NĂM HỌC 2026-2027

TÊN ĐỀ TÀI TIẾNG VIỆT: Thiết kế và xây dựng cơ chế Continuous Training thích ứng với Data Drift trong MLOps dưới ngân sách gán nhãn hạn chế

TÊN ĐỀ TÀI TIẾNG ANH: Design and Implementation of a Continuous Training Mechanism for Data Drift Adaptation in MLOps under a Limited Labeling Budget

Cán bộ hướng dẫn: ThS. Lê Anh Tuấn

Thời gian thực hiện: Từ ngày 07/09/2026 đến ngày 25/12/2026

Sinh viên thực hiện:

Trần Nguyễn Việt Hoàng – 23520541 – MMTT2023.1 – 0828552878

Bùi Ngọc Thái – 23521412 – MMTT2023.3 – 0817008488

Nội dung đề tài:

I. Tổng quan đề tài

1.1. Bối cảnh và vấn đề nghiên cứu

Trong môi trường vận hành thực tế (production), các mô hình học máy (Machine Learning - ML) thường xuyên đối mặt với sự suy giảm hiệu năng do phân phối dữ liệu đầu vào liên tục biến động theo thời gian, bao gồm Data Drift (thay đổi phân phối đặc trưng $P(X)$ và Concept Drift (thay đổi quan hệ xác suất giữa đặc trưng và nhãn $P(Y|X)$). Để duy trì độ chính xác của hệ thống, Continuous Training (CT) là cơ chế thiết yếu giúp tự động hóa quy trình tái huấn luyện và cập nhật mô hình phù hợp với điều kiện vận hành mới.

Tuy nhiên, rào cản cốt lõi khi triển khai Continuous Training trong thực tế là sự khan hiếm nhãn (label scarcity). Đa số các luồng dữ liệu vận hành chỉ nhận được nhãn phản hồi rất trễ hoặc chỉ có một tỷ lệ nhỏ dữ liệu được gán nhãn do chi phí chuyên gia cao. Trong bối cảnh đó, các cơ chế kích hoạt tái huấn luyện hiện hành bộc lộ những điểm nghẽn nghiêm trọng:

- Cảnh báo Data Drift không đồng nghĩa với suy giảm hiệu năng:** Thay đổi phân phối đặc trưng đầu vào có thể xảy ra ở những vùng biên mà mô hình vẫn phân loại chính xác, dẫn đến việc kích hoạt tái huấn luyện giả (false-alarm retraining), gây lãng phí lớn tài nguyên tính toán và chi phí nhân.
- Nguy cơ bỏ sót suy giảm:** Khi quan hệ $P(Y|X)$ thay đổi hoặc phân phối bị dịch chuyển cục bộ nhưng bộ giám sát đầu vào không phát hiện, mô hình có thể bị suy giảm nghiêm trọng mà không có cảnh báo nào được phát ra (silent degradation), dẫn đến tích lũy các dự đoán sai lệch.
- Hạn chế của ước lượng không nhãn:** Các phương pháp ước lượng không nhãn (Unsupervised Performance Estimation) như CBPE hay ATC phụ thuộc chặt chẽ vào

giả định phân phối và mức độ hiệu chuẩn xác suất (confidence calibration). Khi calibration bị lệch trên miền dữ liệu vận hành mới, ước lượng sẽ sai lệch đáng kể.

- **Bài toán đánh đổi chi phí và chất lượng:** Tái huấn luyện quá sớm gây tốn kém compute và ngân sách nhân; can thiệp quá muộn làm suy giảm độ tin cậy của dịch vụ.

Bên cạnh đó, dưới góc độ kỹ thuật trong kiến trúc MLOps, một quyết định kích hoạt tái huấn luyện đúng về mặt lý thuyết vẫn có thể thất bại trong vận hành nếu hệ thống phân tán không xử lý được các sự cố bất đồng bộ: yêu cầu lặp gây cạn kiệt ngân sách nhân, worker/pod lỗi giữa chừng làm mất bước điều phối, callback đến muộn hoặc đảo thứ tự gây cập nhật sai phiên bản, và việc thiếu cơ chế chốt snapshot dữ liệu dẫn đến mất tính tái lập. Do đó, đề tài đặt trọng tâm vào việc thiết kế một cơ chế ra quyết định tối ưu ngân sách nhân đi liền với một hạ tầng điều phối tin cậy, bền vững.

1.2. Cơ sở kế thừa và đóng góp dự kiến

Đề tài kế thừa và tổng hợp có chọn lọc từ các công trình nghiên cứu đo lường cụ thể:

- **ATC (ICLR 2022):** Kế thừa phương pháp học ngưỡng độ tự tin (confidence threshold) trên dữ liệu tham chiếu có nhãn để ước lượng accuracy trên dữ liệu đích không nhãn; sử dụng làm baseline đối chứng nhẹ.
- **CBPE (Machine Learning 2026):** Kế thừa phương pháp ước lượng ma trận nhầm lẫn (confusion matrix) và các chỉ số chất lượng từ xác suất lớp đã qua hiệu chuẩn dưới điều kiện covariate shift; sử dụng làm bộ ước lượng không nhãn chính trong hệ thống.
- **MLDemon (AISTATS 2022):** Kế thừa ý tưởng kết hợp tín hiệu giám sát không nhãn với việc yêu cầu mở nhãn bổ sung có kiểm soát ngân sách để nhận diện suy giảm hiệu năng.
- **Active Testing (ICML 2021):** Vận dụng nguyên lý phân tách nghiêm ngặt giữa tập mẫu dùng để đánh giá mô hình và tập mẫu dùng để huấn luyện, đảm bảo đánh giá không thiên lệch và tiết kiệm nhân.
- **Modyn (SIGMOD 2025):** Kế thừa mô hình kiến trúc pipeline phân tách rạch ròi giữa trigger policy và data-selection policy trong điều phối quy trình học máy hướng dữ liệu.

Đóng góp dự kiến: Đóng góp dự kiến của đề tài tập trung vào tính ứng dụng kỹ thuật và bằng chứng thực nghiệm:

1. Thiết kế và hiện thực hóa một vòng điều phối Continuous Training khép kín với cơ chế kiểm chứng ít nhãn (sample-efficient verification), kết hợp linh hoạt giữa cảnh báo drift, ước lượng không nhãn và lấy mẫu kiểm chứng theo khoảng bất định thống kê.
2. Xây dựng cơ chế Sổ cái ngân sách (Budget Ledger) phân bổ quota độc lập cho kiểm chứng, huấn luyện và thẩm định; đảm bảo tính toàn vẹn trạng thái và chống xử lý lặp thông qua Transactional Outbox Pattern và tiến trình đối soát (reconciliation) trên hạ tầng Kubernetes / Argo Workflows / Kubeflow.
3. Cung cấp bộ thực nghiệm đối chứng toàn diện trên hai bộ dữ liệu thực tế, làm rõ đánh đổi Pareto giữa chất lượng dự đoán, chi phí gán nhãn và tần suất tái huấn luyện (RQ1), đồng thời đo lường khả năng chịu lỗi và thời gian phục hồi khi có sự cố hệ

thống (RQ2).

1.3. Câu hỏi nghiên cứu

RQ1 (Hiệu quả quyết định & Đánh đổi chất lượng - chi phí): Với cùng một giới hạn ngân sách nhân và quy trình huấn luyện cố định, cơ chế kiểm chứng suy giảm bằng nhân thật trước khi tái huấn luyện đạt được mức đánh đổi như thế nào giữa chất lượng dự đoán (error rate, F1), chi phí nhân, số lần huấn luyện và tỷ lệ bỏ sót suy giảm so với các chiến lược: kích hoạt theo data drift thuần túy, kích hoạt theo bộ ước lượng không nhân, hoặc kiểm tra nhân định kỳ theo lịch cố định?

RQ2 (Độ tin cậy vận hành & Khả năng phục hồi điều phối): Vòng điều phối Continuous Training triển khai trên nền tảng Argo Workflows và Kubeflow Training Operator có bảo toàn được hạn mức ngân sách nhân, tính nhất quán dữ liệu/phiên bản và điều kiện cập nhật khi xảy ra các sự cố hệ thống phân tán (sự kiện trùng lặp, worker/pod bị tắt đột ngột, callback đến trễ) không; và hệ thống mất bao lâu (recovery time) để tự động phục hồi sau khi các thành phần lỗi hoạt động trở lại?

II. Mục tiêu của đề tài

2.1. Mục tiêu tổng quát

Thiết kế, xây dựng và đánh giá thực nghiệm một cơ chế điều phối Continuous Training thích ứng với Data Drift có kiểm chứng suy giảm hiệu năng dưới ngân sách nhân hạn chế, đảm bảo tính bền vững trạng thái, độ tin cậy vận hành và tính toàn vẹn phiên bản trên hạ tầng MLOps.

2.2. Mục tiêu cụ thể

1. Xây dựng dịch vụ phục vụ suy luận xác suất lớp đã hiệu chuẩn, luồng thu thập dữ liệu bất đồng bộ và API tiếp nhận nhân phản hồi; tích hợp Evidently AI để giám sát data drift và module CBPE/ATC để ước lượng hiệu năng không nhân.
2. Thiết kế chính sách quyết định 4 trạng thái (KEEP, REQUEST_LABELS, HOLD, RETRAIN) cùng cơ chế Budget Ledger quản lý quota phân bổ độc lập cho 3 mục đích: kiểm chứng, huấn luyện và đánh giá mô hình.
3. Hiện thực hóa quy trình điều phối tái huấn luyện tự động sử dụng Argo Workflows và Kubeflow Training Operator: chốt snapshot dữ liệu, huấn luyện mô hình ứng viên, thẩm định qua Evaluation Gate và cập nhật phiên bản phục vụ có kiểm soát.
4. Xây dựng cơ chế đảm bảo tính nhất quán dữ liệu, chống xử lý lặp (idempotency) và tự động phục hồi sự cố thông qua Transactional Outbox Pattern và tiến trình đối soát nền (reconciliation worker).
5. Tự động hóa triển khai hạ tầng trên AWS / Kubernetes bằng Terraform, Ansible và Argo CD; thiết lập hệ thống quan sát vận hành bằng Prometheus và Grafana.
6. Thực hiện các kịch bản thực nghiệm đối chứng trên 2 bộ dữ liệu dạng bảng công khai và kiểm thử chèn lỗi có kiểm soát (fault injection) để trả lời thấu đáo hai câu hỏi nghiên cứu RQ1 và RQ2.

III. Đối tượng và phạm vi nghiên cứu

3.1. Đối tượng nghiên cứu

- **Quy trình điều phối Continuous Training:** Vòng điều phối Continuous Training

khép kín từ giám sát suy luận, ước lượng chất lượng, kiểm chứng nhãn, tái huấn luyện, thẩm định công chất lượng đến kích hoạt phiên bản mô hình mới trong môi trường MLOps.

- **Quản lý ngân sách nhãn:** Cơ chế phân chia, kiểm soát quota và tối ưu hóa việc sử dụng nhãn hạn chế giữa ba giai đoạn: kiểm chứng mô hình hiện hành, huấn luyện mô hình mới và thẩm định mô hình ứng viên.
- **Độ tin cậy vận hành hệ thống:** Tính toàn vẹn của dữ liệu snapshot, tính lũy đẳng (idempotency) của tác vụ, tính đúng đắn của phiên bản mô hình phục vụ và khả năng tự phục hồi khi gặp các lỗi phân tán.

3.2. Phạm vi và giả định

- **Phạm vi dữ liệu và mô hình:** Dữ liệu dạng bảng (tabular data), bài toán phân loại nhị phân (binary classification) trên hai bộ dữ liệu công khai (ưu tiên có cấu trúc chuỗi thời gian hoặc miền tự nhiên). Áp dụng cho một họ mô hình có xác suất lớp đã hiệu chuẩn (calibrated probabilities), đề xuất bắt đầu từ Logistic Regression / XGBoost.
- **Giả định về nhãn:** Dữ liệu có ground-truth đầy đủ ở phía bộ đánh giá ngoại tuyến (evaluator). Phía hệ thống vận hành chỉ được phép đọc các nhãn được mở theo quota ngân sách sau mỗi batch. Mọi nhãn đã mở đều tính chi phí 1 đơn vị/mẫu và được giả định là chính xác. Yêu cầu mở nhãn được mô phỏng qua API, không nghiên cứu hành vi người gán nhãn.
- **Ranh giới ngoài phạm vi (Out-of-scope):** Không nghiên cứu chẩn đoán nguyên nhân nhân quả (causal root-cause diagnosis); không nghiên cứu học tăng dần (incremental learning), gán nhãn giả (pseudo-labeling) hay tối ưu siêu tham số động; không nghiên cứu huấn luyện phân tán đa GPU, autoscaling cụm lớn hay tính sẵn sàng cao đa vùng (multi-region HA).

IV. Phương pháp thực hiện

4.1. Giám sát và tổ chức dữ liệu

Dữ liệu ban đầu được chia thành ba tập tách biệt nghiêm ngặt: D_{train} dùng huấn luyện mô hình ban đầu (Champion v1); D_{cal} dùng hiệu chuẩn xác suất và huấn luyện ngưỡng cho các bộ ước lượng (CBPE, ATC); D_{ref} dùng làm mốc phân phối tính toán data drift cho Evidently AI và mốc sai số cơ sở e_{ref} .

Trong giai đoạn vận hành, luồng dữ liệu và giám sát được tổ chức như sau:

- **Thu thập log suy luận:** Dịch vụ Model Server tiếp nhận yêu cầu suy luận, ghi nhận đầy đủ bản ghi gồm prediction ID, timestamp, vector đặc trưng X , nhãn dự đoán, xác suất lớp dương $P(Y=1|X)$ đã hiệu chuẩn và phiên bản mô hình phục vụ.
- **Lưu trữ dữ liệu vận hành:** Toàn bộ sự kiện suy luận được phát bất đồng bộ vào Kafka/Redpanda (Message Bus) để đảm bảo độ trễ thấp cho luồng phục vụ. Dịch vụ Consumer Service tiêu thụ sự kiện từ Kafka Topic, lưu trữ có cấu trúc vào PostgreSQL và định kỳ kết xuất các bản chụp batch lên AWS S3.
- **Giám sát Drift và Ước lượng hiệu năng:** Dữ liệu vận hành được gom thành các cửa sổ batch W_t . Dịch vụ Evidently AI định kỳ so sánh phân phối $P(X)$ của W_t với D_{ref} trên S3 bằng các phép kiểm định thống kê (Wasserstein distance, PSI). Đồng thời, thư viện

CBPE tính toán mức sai số ước lượng $\hat{e}_t$ từ xác suất dự đoán; ATC đối chứng ngưỡng tự tin để xác định mức suy giảm ước lượng $\hat{\Delta}_t = \hat{e}_t - e_{ref}$. Tín hiệu cảnh báo từ Evidently hoặc CBPE đóng vai trò kích hoạt bước kiểm chứng, không tự ý kết luận mô hình đã suy giảm.

4.2. Kiểm chứng suy giảm với ngân sách nhân hạn chế

Để khắc phục hạn chế của các bộ ước lượng không nhân và tránh tái huấn luyện giả, đề tài thiết lập cơ chế kiểm chứng bằng nhân thật dưới hạn mức ngân sách chặt chẽ:

- **Sổ cái ngân sách đa mục đích (Budget Ledger):** Thiết lập tổng ngân sách nhân B cho một episode vận hành. Ngân sách được phân bổ và quản lý nghiêm ngặt qua Budget Ledger cho 3 mục đích độc lập: $B_{used} = B_{verify} + B_{train} + B_{gate} \leq B$. Trong đó: B_{verify} dành cho kiểm chứng mô hình hiện hành; B_{train} cung cấp nhân vào tập tái huấn luyện; B_{gate} dành riêng cho thẩm định candidate tại cổng chất lượng.
- **Phân chia mẫu tiền nghiệm (Seed Partitioning):** Tại mỗi batch W_t , trước khi mở bất kỳ nhân nào, một hàm băm có seed cố định sẽ phân chia các mẫu dữ liệu vào 3 pool tách biệt (verify pool, train pool, gate pool). Nhân mở cho mục đích nào chỉ được dùng cho mục đích đó, tuyệt đối không tái sử dụng chéo trong thực nghiệm chính để loại trừ rò rỉ dữ liệu (data leakage) và thiên lệch đánh giá (evaluation bias). Mọi nhân đã mở đều được tính vào ngân sách.
- **Lịch kiểm chứng kết hợp:** Hệ thống duy trì kiểm tra định kỳ (audit schedule k batch) ngay cả khi không có cảnh báo (để phát hiện suy giảm không thể hiện qua phân phối đầu vào), đồng thời kích hoạt kiểm chứng ngay khi có cảnh báo drift hoặc ước lượng chất lượng giảm.
- **Khoảng bất định và quy tắc hai lượt:** Khi kích hoạt, hệ thống lấy ngẫu nhiên m_1 mẫu từ pool kiểm chứng để tính toán mức tăng tỷ lệ lỗi thực tế $\hat{\Delta}_t = \hat{e}_t - e_{ref}$ kèm khoảng tin cậy/bất định thống kê $I_t = [L_t, U_t]$ có xét sai số lấy mẫu ở cả tập production và mốc tham chiếu. Nếu cận dưới $L_t > \delta_{harm}$ (vượt ngưỡng suy giảm có hại): xác nhận mô hình suy giảm thực tế $\rightarrow$ RETRAIN. Nếu cận trên $U_t \leq \delta_{keep}$ (nằm trong ngưỡng chấp nhận): đủ bằng chứng kết luận mô hình hoạt động tốt $\rightarrow$ KEEP. Nếu khoảng I_t giao cắt ngưỡng hoặc estimator bất đồng lớn: cho phép lấy thêm tối đa một lượt bổ sung m_2 mẫu từ cùng hoán vị ngẫu nhiên. Nếu vẫn chưa xác định hoặc hết quota $\rightarrow$ HOLD.

4.3. Chính sách quyết định và tái huấn luyện

Chính sách điều phối đưa ra một trong 4 quyết định cụ thể dựa trên bằng chứng thu thập được:

- **KEEP:** Mô hình hiện hành có bằng chứng chất lượng nằm trong biên chấp nhận; lưu bằng chứng và tiếp tục giám sát.
- **REQUEST_LABELS:** Cần thêm dữ liệu kiểm chứng và còn hạn mức quota B_{verify} ; chốt danh sách ID trước khi mở nhân, ghi nhận chi phí vào Budget Ledger.
- **HOLD:** Tạm hoãn tái huấn luyện khi chưa đủ bằng chứng, thiếu mẫu huấn luyện hoặc cạn kiệt ngân sách; tiếp tục phục vụ bằng champion hiện tại, ghi rõ lý do và thời điểm xét lại.
- **RETRAIN:** Nhân thật xác nhận suy giảm, pool huấn luyện có đủ mẫu và còn quota

cho Evaluation Gate; khởi tạo quy trình tái huấn luyện tự động.

Quy trình Tái huấn luyện (Retraining) và Cổng kiểm soát chất lượng (Evaluation Gate):

- **Chốt snapshot huấn luyện:** Tự động kết xuất Dataset Snapshot từ pool huấn luyện production gần nhất kết hợp dữ liệu lịch sử D_{train} theo tỷ lệ cố định, chốt manifest kèm mã băm SHA-256 lên S3.
- **Huấn luyện và đóng gói:** Mô hình ứng viên (Candidate) được huấn luyện theo recipe cố định và hiệu chuẩn trên tập calibration độc lập.
- **Thẩm định ghép cặp độc lập:** Sau khi build container và đăng ký phiên bản, Candidate bắt buộc phải trải qua Evaluation Gate: so sánh ghép cặp (paired comparison) trực tiếp với Champion trên tập dữ liệu kiểm thử độc lập thuộc pool gate (B_{gate}), đồng thời kiểm tra trên historical holdout để chống suy giảm trí nhớ (catastrophic forgetting).
- **Điều kiện cập nhật phiên bản:** Candidate chỉ được thay thế Champion khi lợi ích vượt trội đạt ngưỡng định trước, không suy giảm trên holdout và mô hình Champion hiện hành trên cụm phục vụ chưa bị thay đổi bởi một tiến trình khác.

4.4. Kiến trúc hệ thống và công cụ triển khai theo từng Workflow

Hệ thống MLOps được thiết kế theo kiến trúc hướng dịch vụ, làm rõ việc sử dụng công cụ và hạ tầng cụ thể cho từng workflow chức năng:

- **Workflow 1:** Model Server (FastAPI) tiếp nhận request suy luận qua REST API, xuất xác suất lớp dương đã hiệu chuẩn; đẩy event suy luận vào Kafka/Redpanda Event Bus; Consumer Service lưu log vào PostgreSQL và đẩy batch lên AWS S3.
- **Workflow 2:** Dịch vụ Evidently AI định kỳ kiểm tra data drift giữa batch production và reference mốc trên S3; thư viện CBPE và ATC tính toán sai số ước lượng và ngưỡng tự tin; Celery Scheduler / Redis kích hoạt kiểm chứng khi có cảnh báo.
- **Workflow 3:** Control Plane (Django) quản lý vòng đời MaintenanceRun; thực thi chính sách quyết định 4 trạng thái; quản lý quota trong Budget Ledger bằng transaction ACID trong PostgreSQL; áp dụng Transactional Outbox Pattern.
- **Workflow 4:** Argo Events nhận webhook kích hoạt; Argo Workflows điều phối pipeline DAG (chốt snapshot dữ liệu, tạo PyTorchJob/TFJob trên Kubernetes qua Kubeflow Training Operator, đóng gói image lên Harbor Registry, chạy pod Evaluation Gate và kích hoạt rollout).
- **Workflow 5:** Terraform tự động cấp phát hạ tầng đám mây (VPC, Subnet, EC2, S3, IAM); Ansible tự động cấu hình OS, cài đặt Kubernetes và khởi tạo Argo CD; Argo CD đồng bộ toàn bộ app/manifest từ Git theo mô hình GitOps.
- **Workflow 6:** Prometheus và ServiceMonitor thu thập metrics thời gian tác vụ, số lượt retry, ngân sách nhân còn lại, mức sử dụng CPU/GPU/RAM; Alertmanager gửi cảnh báo khi workflow kẹt hoặc job thất bại.

4.5. Quản lý trạng thái và cơ chế phục hồi khi lỗi

Mỗi lượt điều phối (MaintenanceRun) được quản lý như một máy trạng thái hữu hạn có định danh duy nhất (UUID), lưu vết đầy đủ mối liên kết với Argo Workflow UID, TrainingJob ID, PyTorchJob UID và phiên bản Champion kỳ vọng. Quy trình chuyển trạng thái tuân thủ 4 nguyên tắc bảo toàn bất biến:

1. **Bảo toàn hạn mức ngân sách (Budget Invariant):** Quota nhận được giữ và chốt trong các transaction cơ sở dữ liệu có khóa; tổng ngân sách đã dùng và đang giữ không bao giờ vượt trần. Mọi yêu cầu nhận đồng thời đều được kiểm tra hạn mức nguyên tử.
2. **Tính lũy đẳng (Idempotency):** Dùng khóa nghiệp vụ (idempotency key) và ràng buộc duy nhất trong DB; các sự kiện retry từ network không tạo ra tác vụ huấn luyện trùng lặp, không trừ tiền hai lần và không đăng ký phiên bản lặp.
3. **Transactional Outbox Pattern & Đối soát nền:** Trạng thái nghiệp vụ và sự kiện điều phối được lưu trong cùng transaction cơ sở dữ liệu. Worker định kỳ đối soát trạng thái giữa PostgreSQL với runtime của Argo Workflows và Kubeflow pod. Khi mất kết nối hoặc mất callback, hệ thống chủ động truy vấn runtime theo UID để đồng bộ trạng thái, không kích hoạt chạy lại mù quáng.
4. **Nhất quán phiên bản mô hình:** Cổng đánh giá Evaluation Gate gắn chặt với Champion, Candidate và Dataset Snapshot cụ thể. Trước khi kích hoạt phiên bản mới, hệ thống kiểm tra lại xem Champion trên cụm phục vụ có bị thay đổi trong quá trình huấn luyện hay không; nếu đã bị thay đổi, Candidate cũ sẽ bị đánh dấu thay thế (superseded) và bị hủy kích hoạt an toàn.

4.6. Thiết kế thực nghiệm chất lượng và chi phí (RQ1)

Thiết kế 6 kịch bản dữ liệu mô phỏng vận hành để kiểm thử tính thích ứng của hệ thống:

1. Ổn định (không drift)
2. Covariate shift với nhiều mức tác động
3. Phân phối quay lại (recurrent drift)
4. Concept drift / mất hiệu chuẩn xác suất
5. Nhận phản hồi có thiên lệch (biased feedback)
6. Hoàn toàn không nhận được nhận bổ sung.

Thực hiện so sánh đối chứng giữa 6 chiến lược kích hoạt có cùng recipe và gate:

- B0 (Giữ nguyên mô hình ban đầu, không retrain)
- B1 (Kích hoạt tái huấn luyện ngay khi có cảnh báo data drift)
- B2 (Kích hoạt tái huấn luyện ngay khi bộ ước lượng CBPE/ATC báo suy giảm)
- B3 (Mở nhận ngẫu nhiên theo lịch định kỳ cố định để kiểm tra và quyết định)
- B4 (Cơ chế đề xuất kết hợp audit định kỳ, tín hiệu cảnh báo và kiểm chứng nhận tối đa 2 lượt theo khoảng bất định)
- B5 (Đối chứng lý tưởng với nhận đầy đủ cho việc giám sát).

Chỉ số đo lường RQ1: Sai số ước lượng (MAE), tỷ lệ kích hoạt sai (false triggers), tỷ lệ bỏ sót suy giảm (missed degradations), tỷ lệ lỗi tích lũy, F1-score, Recall, thời gian đưa chất lượng trở lại biên chấp nhận (recovery time), tổng số nhận tiêu tốn theo mục đích (verify/train/gate), số lần huấn luyện và số lượng candidate bị loại bỏ.

4.7. Kiểm thử độ tin cậy và phục hồi vận hành (RQ2)

Thiết kế bộ kiểm thử chèn lỗi có kiểm soát (Fault Injection Matrix) chạy trực tiếp trên môi trường AWS / Kubernetes:

1. Nhân gửi trùng hoặc request đồng thời khi gần hết quota
2. Worker hoặc pod Control Plane dừng đột ngột sau commit/dispatch
3. Callback từ Argo gửi về trễ, bị lặp hoặc đảo thứ tự
4. Job huấn luyện, container build hoặc gate bị lỗi/timeout
5. Champion trên cụm hoặc dữ liệu S3 thay đổi giữa các bước
6. Candidate không vượt qua Evaluation Gate hoặc container lỗi crash-loop.

Chỉ số đo lường RQ2: Số lần vi phạm ngân sách (yêu cầu = 0), số job/cập nhật trùng lặp (yêu cầu = 0), tỷ lệ quyết định có đầy đủ chuỗi truy vết (yêu cầu = 100%), thời gian phục hồi trung vị và p95 (median & p95 MTTR) từ lúc thành phần lỗi hoạt động trở lại đến khi luồng tiếp tục hoặc kết thúc có kiểm soát.

V. Kết quả dự kiến

5.1. Sản phẩm phần mềm

1. Hệ thống thực nghiệm MLOps triển khai vòng Continuous Training có điều kiện nhằm thích ứng với data drift dưới ngân sách nhân hạn chế.
2. Module quản lý ngân sách nhân (Budget Ledger) và chính sách kiểm chứng suy giảm tích hợp thư viện CBPE, ATC và Evidently AI.
3. Bộ WorkflowTemplate trên Argo Workflows và cấu hình Kubeflow Training Operator phục vụ huấn luyện, đóng gói container, thẩm định cổng chất lượng và cập nhật có kiểm soát.
4. Bộ mã nguồn tự động hóa hạ tầng (Terraform, Ansible) và cấu hình GitOps (Argo CD) có khả năng tái lập môi trường hoàn toàn từ đầu.
5. Hệ thống giám sát vận hành gồm Prometheus metrics, ServiceMonitor và Grafana Dashboards chuyên biệt cho vòng Continuous Training.

5.2. Sản phẩm nghiên cứu

1. Bộ công cụ phát lại dữ liệu (Replay Harness) đánh giá định lượng 6 chiến lược kích hoạt trên 2 tập dữ liệu dạng bảng chuẩn.
2. Bộ dữ liệu nhật ký và báo cáo phân tích định lượng chi tiết trả lời trọn vẹn hai câu hỏi nghiên cứu RQ1 và RQ2.

5.3. Tiêu chí nghiệm thu

1. **Nghiệm thu thực thi:** Toàn bộ quy trình Continuous Training từ giám sát, kiểm chứng nhân, huấn luyện đến cập nhật mô hình chạy thực tế thành công trên môi trường AWS / Kubernetes.
2. **Nghiệm thu độ tin cậy:** Vượt qua 100% các bài kiểm thử bất biến vận hành (không vượt ngân sách, không lặp job, không cập nhật sai phiên bản).
3. **Nghiệm thu truy vết:** Mọi quyết định và sự kiện có đầy đủ dữ liệu truy vết liên kết (data lineage) tra cứu được trên hệ thống.

VI. Kế hoạch thực hiện

Giai đoạn	Thời lượng	Nội dung thực hiện	Phân công
-----------	------------	--------------------	-----------

Giai đoạn 1	Tuần 1 - 4 (07/09/2026 – 04/10/2026)	• Khảo sát tài liệu nghiên cứu (CBPE, ATC, MLDemon, Modyn). • Lựa chọn 2 dataset, model family, recipe và thử nghiệm pilot ngân sách. • Thiết kế kiến trúc, mô hình trạng thái và hợp đồng dữ liệu. • Cấu hình Terraform/Ansible; smoke-test Argo/Kubeflow trên AWS/Kubernetes.	Hoàng: Dữ liệu, khảo sát. Thái: Kiến trúc, hạ tầng.
Giai đoạn 2	Tuần 5 - 8 (05/10/2026 – 01/11/2026)	• Xây dựng module thu thập suy luận, API nhận nhận phản hồi và Số cái ngân sách. • Tích hợp Evidently AI và module ước lượng CBPE/ATC. • Xây dựng bộ công cụ phát lại (Replay Harness) che nhãn. • Kiểm thử unit/integration local bằng Docker Compose.	Hoàng: Module ước lượng, replay. Thái: API, ngân sách, Docker.
Giai đoạn 3	Tuần 9 - 12 (02/11/2026 – 29/11/2026)	• Cài đặt chính sách quyết định 4 trạng thái và các baseline đối chứng. • Xây dựng Argo WorkflowTemplate điều phối training và Evaluation Gate. • Cài đặt Transactional Outbox và worker đối soát phục hồi lỗi. • Xây dựng Grafana Dashboard và cảnh báo Prometheus.	Hoàng: Policy quyết định, gate. Thái: Điều phối Argo, outbox, metrics.
Giai đoạn 4	Tuần 13 - 16 (30/11/2026 – 25/12/2026)	• Chạy thực nghiệm định lượng trên 2 dataset trả lời RQ1 (chất lượng vs chi phí). • Thực hiện các thí nghiệm loại bỏ thành phần (Ablation study). • Chạy bộ kiểm thử chèn lỗi trên AWS/Kubernetes trả lời RQ2 (độ tin cậy & MTTR). • Thu thập số liệu, phân tích các trường hợp thất bại. • Tổng hợp kết quả thực nghiệm, demo, viết báo cáo, chuẩn bị slides bảo vệ khóa luận tốt nghiệp.	Hoàng: Thực nghiệm RQ1, thống kê. Thái: Kiểm thử lỗi RQ2, đo tài nguyên. Cả hai: Tổng hợp kết quả thực nghiệm, demo, báo cáo và slides.

VII. Tài liệu tham khảo

- [1] J. Kivimäki, J. Bialek, W. Kuberski, and J. K. Nurminen, "Performance Estimation in Binary Classification Using Calibrated Confidence," Machine Learning, vol. 115, Art. 67, 2026. DOI: 10.1007/s10994-025-06970-3.
- [2] S. Garg, S. Balakrishnan, Z. C. Lipton, B. Neyshabur, and H. Sedghi, "Leveraging Unlabeled Data to Predict Out-of-Distribution Performance," in International Conference

on Learning Representations (ICLR), 2022. <https://arxiv.org/abs/2201.04234>.

- [3] T. Ginart, M. J. Zhang, and J. Zou, "MLDemon: Deployment Monitoring for Machine Learning Systems," in Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS), PMLR, vol. 151, pp. 3962–3997, 2022. <https://proceedings.mlr.press/v151/ginart22a.html>.
- [4] J. Kossen, S. Farquhar, Y. Gal, and T. Rainforth, "Active Testing: Sample-Efficient Model Evaluation," in Proceedings of the International Conference on Machine Learning (ICML), PMLR, vol. 139, pp. 5753–5763, 2021. <https://proceedings.mlr.press/v139/kossen21a.html>.
- [5] M. Böther, T. Robroek, V. Gsteiger, R. Holzinger, X. Ma, P. Tözün, and A. Klimovic, "Modyn: Data-Centric Machine Learning Pipeline Orchestration," Proceedings of the ACM on Management of Data (SIGMOD), vol. 3, no. 1, Art. 55, 2025. DOI: 10.1145/3709705.
- [6] J. Białek, J. Kivimäki, W. Kuberski, and N. Perrakis, "Estimating Model Performance Under Covariate Shift Without Labels," in Advances in Neural Information Processing Systems (NeurIPS), vol. 38, 2025. <https://arxiv.org/abs/2401.08348>.
- [7] Evidently AI, "Data Drift Detection and Production ML Monitoring," Technical Documentation, 2026. https://docs.evidentlyai.com/metrics/explainer_drift. Truy cập ngày 07/09/2026.

Xác nhận của CBHD
(Ký tên và ghi rõ họ tên)

Lê Anh Tuấn

TP. HCM, ngày 14 tháng 09 năm 2026
Sinh viên thực hiện
(Ký tên và ghi rõ họ tên)

Trần Nguyễn Việt Hoàng Bùi Ngọc Thái